

INFORME DE LA ACTIVIDAD EVALUABLE SOBRE

PROGRAMACIÓN PYTHON

Máster Universitario en Bioinformática (Universidad Alfonso X el Sabio)

Nombre del estudiante: Antonio Bellido Maya

Asignatura: Fundamentos de programación para titulaciones de ciencias de la salud

ÍNDICE

1. Introducción y alcance
2. Descripción general del funcionamiento del sistema
3. Estructuras de datos empleadas
4. Estructuras de control de flujo empleadas
5. Control de excepciones y errores
6. Casos particulares y tasa de éxito
7. Dificultades encontradas
8. Conclusiones

1. Introducción y alcance

El objetivo de este proyecto es desarrollar un programa en Python capaz de traducir secuencias de RNA mensajero a secuencias de proteínas, utilizando archivos en formato FASTA como entrada.

Esta actividad se enmarca dentro de la asignatura *Fundamentos de programación para titulaciones de ciencias* y tiene como finalidad poner en práctica los conocimientos básicos adquiridos sobre programación en Python, aplicados a un contexto bioinformático real.

El programa desarrollado permite:

- Leer archivos FASTA que contienen secuencias de RNA mensajero.
- Eliminar la línea identificadora propia del formato FASTA.
- Traducir la secuencia de RNA a una secuencia de aminoácidos utilizando el código genético.
- Gestionar codones desconocidos.
- Guardar los resultados obtenidos en un archivo binario y leerlos posteriormente.

El alcance del proyecto se limita a la traducción básica de RNA a proteína, sin considerar aspectos más avanzados como marcos de lectura alternativos o análisis funcional.

Este proyecto se ha planteado como una primera aproximación al uso de la programación en Python aplicada a problemas sencillos de bioinformática. En este contexto, se ha buscado priorizar la claridad del código, la correcta organización del proyecto y la comprensión del flujo general del programa, por encima de la complejidad algorítmica.

El desarrollo del programa se ha realizado de manera incremental, comprobando el funcionamiento de cada parte antes de avanzar al siguiente paso. Esta metodología ha permitido detectar errores de forma temprana y comprender mejor la relación entre los datos biológicos de entrada y los resultados obtenidos tras su procesamiento.

2. Descripción general del funcionamiento del sistema

El sistema desarrollado sigue un flujo de trabajo secuencial claramente definido, desde la lectura de los datos de entrada hasta la generación de los resultados finales. En primer lugar, el programa principal comprueba la existencia y validez del archivo FASTA proporcionado como entrada. Una vez verificado, se procede a la lectura completa del contenido del archivo.

A continuación, se elimina la línea correspondiente al identificador FASTA y se extrae la secuencia de RNA mensajero, que se limpia para evitar errores derivados de espacios o saltos de línea. Esta secuencia se envía posteriormente a un módulo independiente encargado de realizar la traducción a proteína utilizando un diccionario del código genético.

Finalmente, la secuencia de aminoácidos obtenida se muestra por pantalla y se guarda en un archivo de salida en formato binario, cuyo contenido es leído posteriormente para verificar la correcta escritura de los resultados. Este flujo garantiza una separación clara de responsabilidades y facilita la comprensión global del sistema.

3. Estructuras de datos empleadas

Durante el desarrollo del proyecto se han utilizado varias estructuras de datos básicas de Python, seleccionadas en función de su adecuación al problema planteado.

En primer lugar, se ha empleado un **diccionario** para representar el código genético, donde cada clave corresponde a un codón de RNA (triplete de nucleótidos) y cada valor representa el aminoácido codificado. Esta estructura permite establecer una correspondencia directa entre codones y aminoácidos, facilitando una búsqueda rápida y eficiente durante el proceso de traducción y evitando el uso de múltiples estructuras condicionales.

Las secuencias de RNA y las proteínas resultantes se han manejado como **cadenas de texto (strings)**, lo que permite trabajar de forma sencilla con operaciones habituales

como el corte de subcadenas, la concatenación y la limpieza de caracteres no deseados. Estas operaciones son especialmente frecuentes en bioinformática, donde gran parte de los datos se representan en formato texto.

Por último, se han utilizado **listas** para dividir el contenido del archivo FASTA en líneas, lo que permite identificar y eliminar de forma sencilla la línea correspondiente al identificador del archivo antes de procesar la secuencia biológica.

En conjunto, estas estructuras resultan adecuadas para un problema de este tipo debido a su sencillez, eficiencia y facilidad de uso.

4. Estructuras de control de flujo empleadas

El programa hace uso de diferentes estructuras de control de flujo:

- **Condicionales (if):**

Se emplean para:

- Comprobar la existencia y validez del archivo FASTA.
- Ignorar la línea identificadora del archivo FASTA.
- Gestionar codones desconocidos durante la traducción.

- **Bucles (for):**

Se utilizan para:

- Recorrer las líneas del archivo FASTA.
- Procesar la secuencia de RNA de tres en tres nucleótidos (codones).
- Construir la secuencia de proteína aminoácido a aminoácido.

- **Funciones:**

La lógica de traducción se encapsula en una función definida dentro de un módulo independiente, lo que mejora la reutilización del código y la claridad del programa.

Los bucles empleados en el programa permiten recorrer de forma sistemática tanto las líneas del archivo FASTA como la secuencia de RNA. En particular, el recorrido de la secuencia en pasos de tres nucleótidos garantiza que la lectura se realice correctamente por codones, respetando la estructura del código genético.

El uso de funciones contribuye a una mejor organización del código, separando la lógica de traducción del resto del programa. Esta separación facilita la reutilización de la función de traducción en otros contextos y mejora la legibilidad general del proyecto.

5. Control de excepciones y errores

Para garantizar un funcionamiento correcto del programa, se han tenido en cuenta diferentes situaciones que pueden provocar errores:

- Comprobación de la existencia del archivo FASTA mediante el uso del módulo `os`.
- Verificación de que la ruta proporcionada corresponde a un archivo válido.
- Gestión de codones no contemplados en el diccionario del código genético, asignándoles el valor "X" según lo indicado en el enunciado de la actividad.
- Limpieza de la secuencia de RNA eliminando posibles espacios o saltos de línea que pudieran afectar a la traducción.

Estas medidas permiten que el programa sea más robusto frente a entradas no esperadas.

Aunque no se ha hecho uso explícito de bloques `try/except`, el programa incorpora diferentes mecanismos de prevención de errores que permiten controlar situaciones problemáticas de forma anticipada. Entre estas medidas se incluyen la comprobación de la existencia del archivo de entrada y la validación de su formato antes de proceder a su lectura.

Además, la gestión de codones desconocidos mediante la asignación del carácter "X" permite que el programa continúe su ejecución sin interrumpirse ante datos no

contemplados en el diccionario, aumentando así la robustez del sistema frente a entradas imperfectas.

6. Casos particulares y tasa de éxito

Para validar el funcionamiento del programa se han realizado distintas pruebas utilizando archivos FASTA de ejemplo creados manualmente siguiendo el formato descrito en el enunciado.

Se han probado:

- Secuencias con longitud múltiplo de tres.
- Secuencias que contienen codones no incluidos en el diccionario.
- Secuencias que incluyen codones de parada (STOP).

En todos los casos, el programa ha generado una salida coherente, traduciendo correctamente los codones conocidos y representando los desconocidos mediante el carácter "X". La tasa de éxito en las pruebas realizadas ha sido del 100 %.

Durante la fase de pruebas, se utilizaron secuencias de RNA de distinta longitud y composición con el objetivo de evaluar el comportamiento del programa en situaciones variadas. En particular, se prestó especial atención a la presencia de codones de parada y a la aparición de codones no incluidos en el diccionario del código genético.

El análisis de los resultados obtenidos permitió verificar que el programa traduce correctamente los codones conocidos y gestiona de forma adecuada los casos particulares, manteniendo un comportamiento consistente en todas las pruebas realizadas.

7. Dificultades encontradas

Durante el desarrollo del proyecto se encontraron algunas dificultades, principalmente relacionadas con:

- La gestión de rutas y extensiones de archivos en el sistema operativo.
- La depuración de errores derivados de caracteres invisibles o errores en las secuencias FASTA.
- La necesidad de recargar módulos en el entorno Jupyter Notebook tras realizar modificaciones.

Estas dificultades fueron resueltas mediante pruebas iterativas y una revisión cuidadosa de los datos de entrada.

Una de las principales dificultades encontradas estuvo relacionada con la gestión de archivos en el entorno de desarrollo, especialmente en lo relativo a las extensiones de archivo y las rutas de acceso. Estas dificultades son habituales en las primeras etapas del aprendizaje de la programación y se resolvieron mediante la comprobación sistemática del entorno de trabajo.

Asimismo, se identificó la necesidad de limpiar adecuadamente las secuencias de entrada para evitar errores derivados de caracteres invisibles o saltos de línea inesperados, lo que puso de manifiesto la importancia de la validación de los datos en aplicaciones bioinformáticas.

8. Conclusiones

En este proyecto se ha desarrollado con éxito un programa funcional en Python para la traducción de RNA a proteína a partir de archivos FASTA.

La actividad ha permitido aplicar conceptos fundamentales de programación, como el uso de diccionarios, funciones, estructuras de control y manejo de archivos, en un contexto bioinformático real.

Este trabajo ha servido como una primera aproximación práctica a la bioinformática, sentando las bases para abordar problemas más complejos en el futuro y reforzando la importancia de la correcta gestión de datos biológicos y la programación modular.

La realización de esta actividad ha permitido afianzar conceptos fundamentales de programación en Python, así como comprender mejor el flujo de trabajo típico en proyectos de bioinformática. La traducción de RNA a proteína constituye un ejemplo sencillo pero representativo de cómo la programación puede utilizarse para resolver problemas biológicos reales.

Como trabajo futuro, el programa podría ampliarse incorporando un diccionario completo del código genético, el tratamiento de marcos de lectura alternativos o la lectura de archivos FASTA con múltiples secuencias. No obstante, el alcance actual del proyecto resulta adecuado para los objetivos planteados en la asignatura.